Design and Analysis of Algorithms: CI43

Unit-IV

Dynamic Programming

Dynamic Programming

Dynamic Programming (DP) is a method for solving complex problems by breaking them down into simpler subproblems and solving each subproblem only once, storing its result for future use (this is called memoization).

Dynamic Programming

🔑 **Key Concepts of Dynamic Programming**:

1. **Overlapping Subproblems**:

The problem can be broken down into subproblems that are reused multiple times.

Example: In computing Fibonacci numbers.

2. **Optimal Substructure:**

The optimal solution to the problem can be constructed from the optimal solutions of its subproblems.

Example: **In the knapsack problem**, the best way to fill a knapsack of weight W may depend on the best way to fill a smaller knapsack of weight W - wi.

3. **Memoization or Tabulation**:

- Memoization (Top-Down): Solving recursively and caching results.
- Tabulation (Bottom-Up): Building a table from base cases upward.

Dynamic Programming

- **Example: 0/1 Knapsack Problem**
- In 0/1 Knapsack, you want to choose items to maximize profit without exceeding the weight limit.
  Dynamic programming solves this by building a table of the best profit for each subproblem (e.g., max profit with i items and weight w).

Weighted Interval Scheduling

Weighted Interval Scheduling – Recursive Procedure (Simple Version)

1. Sort the jobs by end time.

2. For each job j, find the last job that doesn't overlap with job j. Call this p(j).

3. Define a function OPT(j) to find the maximum weight using the first j jobs:

$OPT(j)=\max(\text{weight}[j]+OPT(p(j)), OPT(j-1))$

Include job j → Add its weight + best solution before it (OPT(p(j)))

Exclude job j → Just take OPT(j-1)

4.Use recursion to compute OPT(n) where n is the number of jobs.

Weighted Interval Scheduling: A Recursive Procedure

Weighted Interval Scheduling: A Recursive Procedure

Goal:

Select a set of non-overlapping jobs with maximum total weight.

Step-by-step: Designing a Recursive Algorithm

✅ Step 1: Sort jobs

Sort all jobs by their end time.

✅ Step 2: Find p(j)

For each job j, find the last job that finishes before job j starts. Call this p(j).

✅ Step 3: Define the recursive function

Let OPT(j) be the maximum weight for the first j jobs.

Weighted Interval Scheduling: A Recursive Procedure

We have two choices:

Include job j → total = weight[j] + OPT(p(j))

Exclude job j → total = OPT(j-1)

So, the recursive formula is:

$OPT(j)=\max(weight[j]+OPT(p(j)),OPT(j-1))$

✅ Step 4: Base case

If j = 0 (no jobs), then OPT(0) = 0.

Weighted Interval Scheduling: A Recursive Procedure

Steps to Design the Recursive Algorithm

1.Sort the jobs by their end times.

2.Find p(j) for each job j

→ This is the index of the last job that finishes before job j starts.

3.Define a recursive function OPT(j)

→ It gives the maximum weight using the first j jobs.

4.Base Case

→ If j == 0, return 0 (no jobs, no weight).

Weighted Interval Scheduling: A Recursive Procedure

5.Recursive Case

→ At each step, choose the best of:

Include job j: weight[j] + OPT(p(j))

Exclude job j: OPT(j - 1)

→ Return the maximum of these two.

6.Final Answer

→ Compute OPT(n), where n is the total number of jobs.

Weighted Interval Scheduling: A Recursive Procedure

- Problem:Identify and compute the optimal set of intervals with maximum value for the set of intervals and weights given in Table 8c.

Table 8c Weighted Interval

Interval	Start time	Finish time	Weight
1	1	6	10
2	2	4	3
3	5	9	4
4	7	14	20
5	11	18	2

- **Weighted Interval Scheduling Problem** using **Dynamic Programming** for the following intervals:

Interval		Start	Finish	Weight
1	1	6	10	
2	2	4	3	
3	5	9	4	
4	7	14	20	
5	11	18	2	

Step 1: Sort Intervals by Finish Time

Sorted list based on finish times:

Sorted Index	Original Interval	Start	Finish	Weight
2	2	2	4	3
1	1	1	6	10
3	3	5	9	4
4	4	7	14	20
5	5	11	18	2

Weighted Interval Scheduling

Step 2: Compute p(j) (latest non-conflicting interval)

Let p(j) be the index of the last interval (before j) that does not overlap with interval j.

j (Sorted Index)	Interval	Start	Finish	p(j)
1	2	2	4	0
2	1	1	6	0
3	3	5	9	1
4	4	7	14	1
5	5	11	18	3

✅ Step 3: Dynamic Programming Recurrence

Let OPT(j) be the maximum weight for the first j intervals.

Recursive formula:

$OPT(j)=max(weight[j]+OPT(p(j)), OPT(j-1))$

Initialize:

- OPT(0) = 0

Weighted Interval Scheduling

Now compute:

OPT(1) = max(3 + OPT(0), OPT(0)) = max(3, 0) = 3

OPT(2) = max(10 + OPT(0), OPT(1)) = max(10, 3) = 10

OPT(3) = max(4 + OPT(1), OPT(2)) = max(4 + 3, 10) = max(7, 10) = 10

OPT(4) = max(20 + OPT(1), OPT(3)) = max(20 + 3, 10) = max(23, 10) = 23

OPT(5) = max(2 + OPT(3), OPT(4)) = max(2 + 10, 23) = max(12, 23) = 23

✅ Optimal Value = 23

Weighted Interval Scheduling

✅ Step 4: Find the Optimal Set of Intervals (Backtracking)

We backtrack from OPT(5):

- OPT(5) = OPT(4) ⟶ Interval 5 not included
- OPT(4) = weight(Interval 4) + OPT(1) ⟶ Include Interval 4
- OPT(1) = weight(Interval 2) + OPT(0) ⟶ Include Interval 2

✅ Optimal Set of Intervals: Interval 2 and Interval 4

- Interval 2: (2, 4), Weight = 3
- Interval 4: (7, 14), Weight = 20
- Total weight = 23

✅ Final Answer:

- Maximum Total Weight: 23
- Optimal Set of Intervals: Interval 2 and Interval 4

Subset Sums and Knapsacks: Adding a Variable

Subset Sums and Knapsacks: Adding a Variable

The Problem:

Given a set of items, each with a weight and possibly a value, find a subset whose total weight (and possibly total value) satisfies certain constraints.

Subset Sum Problem:

Find if there is a subset whose sum is equal to a target W.

0/1 Knapsack Problem:

Given a weight limit W, select items with weight and value to maximize total value without exceeding W.

Designing the Algorithm:

Use Dynamic Programming.

Define dp[i][w] as the maximum value using the first i items with weight limit w.

Note: dp---Means Dynamic Programming

Subset Sums and Knapsacks: Adding a Variable

Recursive relation:

```
dp[i][w] = max(
            dp[i-1][w],                    // don't take item i
    value[i] + dp[i-1][w - weight[i]]  // take item i if weight[i] ≤ w
)
```

✅ 2. Shortest Paths in a Graph

The Problem:

Given a graph (with positive or negative edge weights), find the shortest distance from a source node to all other nodes.

Designing the Algorithm:

There are different algorithms based on the graph type:

Subset Sums and Knapsacks: Adding a Variable

Dijkstra's Algorithm (for non-negative weights):

Use a priority queue (min-heap) to greedily select the next closest node.

Time: $O((V + E) \log V)$

Bellman-Ford Algorithm (for negative weights):

Relax all edges up to V-1 times.

Detects negative-weight cycles.
Time: $O(V \times E)$

Floyd-Warshall Algorithm (for all-pairs shortest paths):

Dynamic programming.

Time: $O(V^3)$

Subset Sums and Knapsacks: Adding a Variable

3. The Maximum-Flow Problem

The Problem:

Given a flow network with capacities, find the maximum amount of flow from source s to sink t.

Designing the Algorithm:

Use Ford-Fulkerson Method:

1. Start with zero flow.
2. While there is a path from s to t with available capacity (residual capacity > 0), augment flow along this path.
3. Repeat until no more augmenting paths.

Edmonds-Karp Algorithm is an implementation using BFS for finding paths.

Time: $O(V \times E^2)$

1. Knapsack Problem Using Dynamic Programming

We are given:

Item	Weight	Profit
1	2	10
2	3	42
3	4	46

Knapsack capacity W = 6

Let n = 3 (number of items).

We create a DP table K[n+1][W+1] where K[i][w] represents the maximum profit for the first i items with weight capacity w.

Knapsack Problem

Step-by-step DP Table Construction

Initialize:

- Rows = items (0 to 3)
- Columns = weight capacity (0 to 6)
- All K[0][w] = 0 and K[i][0] = 0

Now, fill the table using the 0/1 knapsack recurrence:

K[i][w] = max(K[i-1][w], profit[i-1] + K[i-1][w - weight[i-1]]) if weight[i-1] <= w

K[i-1][w] otherwise

i	w=0	w=1	w=2	w=3	w=4	w=5	w=6
0	0	0	0	0	0	0	0
1	0	0	10	10	10	10	10
2	0	0	10	42	42	52	52
3	0	0	10	42	46	52	52

Result:

Maximum subset-sum (profit) for W=6 = 52

Selected items:

- Item 2 (weight=3, profit=42)
- Item 1 (weight=2, profit=10)

Knapsack Problem

b) Weighted Interval Scheduling Problem

Problem Statement:

Given a set of jobs, each with:

- **start time s_i**
- **finish time f_i**
- **weight/profit v_i**

Choose a subset of non-overlapping jobs to maximize total profit.

Example:

Job	Start	Finish	Profit
1	1	3	5
2	2	5	6
3	4	6	5
4	6	7	4
5	5	8	11
6	7	9	2

Knapsack Problem

Step-by-Step Approach:

1. Sort jobs by finish time.
2. Compute p(j): latest non-conflicting job before job j.
3. Recursive relation (DP):

OPT(j) = max(v_j + OPT(p(j)), OPT(j-1))

4. Base case: OPT(0) = 0

---

p(j) Table:

Job	Finish	p(j)
1	3	0
2	5	0
3	6	1
4	7	3
5	8	2
6	9	4

Note: OPT-Means Optimal Solution

Knapsack Problem

DP Computation:

OPT(0) = 0

OPT(1) = max(5 + OPT(0), OPT(0)) = 5

OPT(2) = max(6 + OPT(0), OPT(1)) = 6

OPT(3) = max(5 + OPT(1), OPT(2)) = 10

OPT(4) = max(4 + OPT(3), OPT(3)) = 14

OPT(5) = max(11 + OPT(2), OPT(4)) = 17

OPT(6) = max(2 + OPT(4), OPT(5)) = 17

Maximum Profit = 17

Jobs selected: 3, 4, 5 or another optimal combination depending on selection order.

Knapsack Problem

Problem 2: To solve the 0/1 Knapsack Problem with the given data:

- **Knapsack Capacity, W = 10**
- **Item weights: w = [4, 7, 5, 3]**
- **Item profits: p = [40, 42, 25, 12]**

We use Dynamic Programming to find the maximum profit and the items included.

Step 1: Dynamic Programming Table Initialization

Let dp[i][w] represent the maximum profit using the first i items with capacity w.

We will fill a table of size (n+1) x (W+1) where n = 4.

Step 2: Build DP Table

We iterate over each item and for each capacity from 0 to W.

Item (i)	Weight (w_i)	Profit (p_i)
1	4	40
2	7	42
3	5	25
4	3	12

Knapsack Problem

Step 3: Final DP Table and Maximum Profit

After running the DP algorithm (code can be shared if needed), the maximum profit for W = 10 is:

✅ **Maximum Profit: 65**

✅ **Items in Knapsack: Item 1 (weight 4, profit 40) and Item 3 (weight 5, profit 25)**

Total weight = 4 + 5 = 9 (within capacity)

Total profit = 40 + 25 = 65

General principals of Dynamic Programming

General Principles of Dynamic Programming

Dynamic Programming (DP) is a problem-solving method used when a problem can be broken down into simpler subproblems that repeat.

🌟 General Principles of DP:

1. Optimal Substructure:

A problem has an optimal substructure if the optimal solution can be built from optimal solutions of its subproblems.

Example: In shortest path problems, the shortest path to a destination goes through shortest paths to intermediate points.

2. Overlapping Subproblems:

The problem can be divided into subproblems which are solved multiple times.

DP solves each subproblem once and stores the result.

3. Memoization and Tabulation:

- **Memoization: Top-down approach (recursive + cache)**
- **Tabulation: Bottom-up approach (iterative)**

4. Stages: Identify each decision point or step (stage) in solving the problem.

Knapsack Problem

5. **States:**

Represent the different situations the problem can be in at each stage.

6. **State Transition:**

A recurrence relation or formula to move from one state to another.

✅ **Example Applications:**

- **Knapsack Problem**
- **Fibonacci Series**
- **Shortest Path (e.g., Bellman-Ford)**
- **Matrix Chain Multiplication**
- **Longest Common Subsequence**

Shortest Path using Floyd's Algorithm

Solve the all-pairs shortest-path problem using Floyd's algorithm for the digraph with the weight matrix given in Fig

$$\begin{bmatrix} 0 & 2 & \infty & 1 & 8 \\ 6 & 0 & 3 & 2 & \infty \\ \infty & \infty & 0 & 4 & \infty \\ \infty & \infty & 2 & 0 & 3 \\ 3 & \infty & \infty & \infty & 0 \end{bmatrix}$$

Initial Weight Matrix W

From \ To	0	1	2	3	4
0	0	2	∞	1	8
1	6	0	3	2	∞
2	∞	∞	0	4	∞
3	∞	∞	2	0	3
4	3	∞	∞	∞	0

Step-by-Step Using Floyd's Algorithm

We will apply:

for k = 0 to 4:

    for i = 0 to 4:

        for j = 0 to 4:

            D[i][j] = min(D[i][j], D[i][k] + D[k][j])

We'll perform 5 iterations (one for each intermediate vertex k). To keep things clear and focused, here's the final shortest-path matrix after all iterations:

- The **Floyd-Warshall** algorithm looks at all pairs of nodes and tries to find **shorter paths** using other nodes as intermediates.
- We start with the direct distances (given in the matrix).
- We then check for each intermediate node if it creates a shorter path for any pair of nodes.
- For example:
- Initially, the distance from **0 to 2** is ∞, but through **0** → **3** → **2** (with weights 1 + 2 = 3), we find a shorter path.

- **Step-by-Step Process**
- **First iteration (k = 0)**: We use node **0** as an intermediate node and update the distances.
- **Second iteration (k = 1)**: Now, we use node **1** as an intermediate node.
- We repeat this for **k = 2, 3, 4**.
- Each iteration updates the matrix by finding shorter paths using the current intermediate node

- **Step 1: k = 0**

- We use **node 0** as an intermediate node to check if any path can be shortened by going through node 0.

- For example:

- **From 0 to 2**: There is no direct path (∞), but we can go fr
  2 to 3.

- **From 1 to 4**: There is no direct path (∞), but we can go fr
  4 to 5.

- After **Step 1 (k = 0)**, the updated matrix looks like this:

From \ To	0	1	2	3	4
0	0	2	3	1	8
1	6	0	3	2	5
2	∞	∞	0	4	∞
3	∞	∞	2	0	3
4	3	∞	6	∞	0

Step 2: k = 1

Next, we use node 1 as an intermediate node.

From \ To	0	1	2	3	4
0	0	2	3	1	7
1	6	0	3	2	5
2	∞	∞	0	4	∞
3	∞	∞	2	0	3
4	5	5	6	∞	0

- From 0 to 4: Previously, we had 8 as the distance from 0 to 4. Now, we can go from 0 → 1 → 4, which costs 2 + 5 = 7. So, we update the distance from 0 to 4 to 7.
- From 4 to 1: There was no direct path before (∞), but we can go from 4 → 0 → 1, which costs 3 + 2 = 5. So, we update the distance from 4 to 1 to 5.

From \ To	0	1	2	3	4
0	0	2	3	1	7
1	6	0	3	2	5
2	∞	∞	0	5	∞
3	∞	∞	2	0	3
4	3	5	6	4	0

- **Step 3: k = 2**
- Now, we use **node 2** as an intermediate node.
- **From 0 to 4**: No update needed.
- **From 1 to 4**: No update needed.
- **From 2 to 3**: The current distance from 2 to 3 is 4, but we can go from 2 ⟶ 0 ⟶ 3, which costs 4 + 1 = 5. So, we update the distance from 2 to 3 to 5.

Step 4: k = 3

From \ To	0	1	2	3	4
0	0	2	3	1	4
1	3	0	3	2	5
2	∞	∞	0	4	∞
3	∞	∞	2	0	3
4	3	5	6	4	0

- Now, we use **node 3** as an intermediate node.
- **From 0 to 4**: The current distance from 0 to 4 is 7, but we can go . So, we update the distance from 0 to 4 to 4.
- **From 1 to 0**: There was no direct path before (∞), but we can go from 1 ⟶ 3 ⟶ 0, which costs 2 + 1 = 3. So, we update the distance from 1 to 0 to 3.

- **Step 5: k = 4**

- Finally, we use **node 4** as an intermediate node.

- **From 2 to 1**: There was no direct path before (∞), but we can go from 2 → 4 → 1, which costs 6 + 5 = 12. So, we update the distance from 2 to 1 to 12.

- **From 2 to 3**: There was no direct path before (∞), but we can go from 2 → 4 → 3, which costs 6 + 4 = 10. So, we

From \ To	0	1	2	3	4
0	0	2	3	1	4
1	3	0	3	2	5
2	10	12	0	10	7
3	5	7	2	0	3
4	3	5	8	4	0

Final All-Pairs Shortest Path Matrix

From \ To	0	1	2	3	4
0	0	2	3	1	4
1	6	0	5	2	5
2	10	12	0	10	7
3	5	7	2	0	3
4	3	5	6	4	0

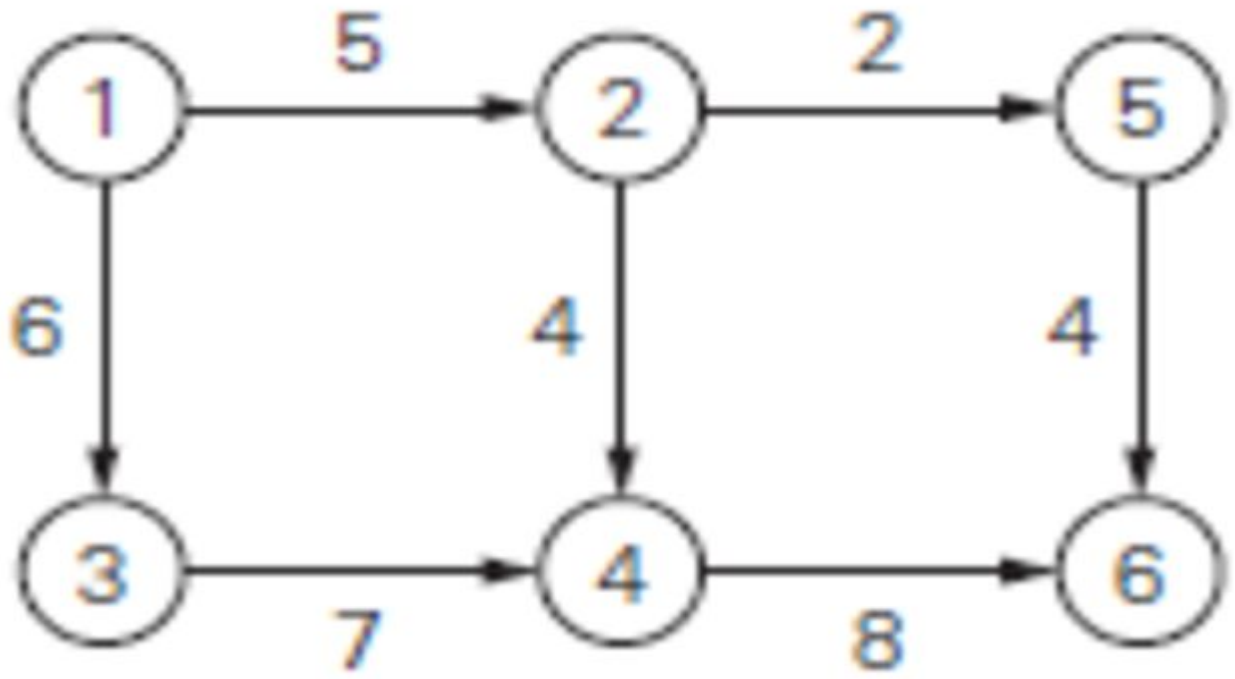
Explanation of a Few Entries:

- **From 0 to 2**: The shortest path is now 3, which is 0 ⟶ 3 ⟶ 2.
- **From 1 to 4**: The shortest path is now 5, which is 1 ⟶ 3 ⟶ 4.
- **From 4 to 2**: The shortest path is now 6, which is 4 ⟶ 0 ⟶ 3 ⟶ 2.
-

Problem:Apply the shortest-augmenting path algorithm to find a maximum flow and a minimum cut in the network given in Fig. Compute the Maximum Flow

Maximum flow refers to the **greatest possible amount of flow** that can move from a **source node (s)** to a **sink node (t)** in a **flow network**, without violating the **capacity constraints** on the edges.

The **minimum cut** is a way of **separating the source (s)** from the **sink (t)** in a flow network by **removing the least total capacity of edges**, such that **no more flow can pass** from source to sink.



Find **Maximum Flow** from **Source = Node 1** to **Sink = Node 6**

Step 1: List the edges with capacities

From → To	Capacity
1 → 2	5
1 → 3	6
2 → 4	4
2 → 5	2
3 → 4	7
4 → 6	8
5 → 6	4

Step 2: Find Augmenting Paths (Shortest Paths from 1 to 6)

👉 **Path 1: 1 → 2 → 5 → 6**

Min capacity = min(5, 2, 4) = **2**

Add 2 units of flow

👉 **Path 2: 1 → 2 → 4 → 6**

Left capacity on 1→2 = 3

Min capacity = min(3, 4, 8) = **3**

Add 3 units of flow

👉 **Path 3: 1 → 3 → 4 → 6**

All edges have enough capacity

Min capacity = min(6, 7, 5) = **5**

Add 5 units of flow

Step 3: Add up all flows

- **Path 1 = 2**
- **Path 2 = 3**
- **Path 3 = 5**

✅ **Total Maximum Flow = 2 + 3 + 5 = 10**

✂️ **Step 4: Minimum Cut (Just the edges from reachable to unreachable)**

After flow is complete, nodes reachable from node 1:

- **1 and 3 (others are blocked due to full capacity)**

So cut edges from:

- **1 → 2**
- **3 → 4**

✅ **Minimum Cut = edges (1→2) and (3→4)**

✅ **Final Answer**

- **Maximum Flow = 10**
- **Minimum Cut = (1→2), (3→4)**

Bellman-Ford Algorithm

The **Bellman-Ford algorithm** is a shortest-path algorithm used in graph theory to compute the shortest path from a single source vertex to all other vertices in a weighted graph. Unlike Dijkstra's algorithm, Bellman-Ford works even when **some edge weights are negative**, making it more versatile in certain applications.

- **Key Features:**
- Handles **negative weight edges**
- Detects **negative weight cycles**
- Time complexity: **O(V × E)** (V = vertices, E = edges)
- Works on **directed** and **undirected** graphs (if all edges are properly directed)

Bellman-Ford Algorithm

Algorithm Steps:

Given a graph G(V, E) and a source vertex src:

1. Initialize distances from the source to all vertices as ∞ and distance to source as 0.

dist[src] = 0

dist[v] = ∞ for all v ≠ src

2. Relax all edges |V| - 1 times:

For each edge (u, v) with weight w:

if dist[u] + w < dist[v]:

    dist[v] = dist[u] + w

3. Check for negative-weight cycles:

    One more pass through all edges. If any distance can still be updated, a negative cycle exists.

if dist[u] + w < dist[v]:

    Report "Negative weight cycle detected"

Bellman-Ford Algorithm

Edge (u→v)	Weight
A → B	4
A → C	5
B → C	3
C → D	4

Source = A

Bellman-Ford Algorithm Steps:

1. **Initialization:**

   Set the distance to the source vertex A as 0 (distance[A] = 0).

   Set the distance to all other vertices as infinity (distance[B] = ∞, distance[C] = ∞, distance[D] = ∞).

2. **Relax the edges (V - 1) times:**

   For each edge, if the distance to the destination vertex can be minimized (by going through the source vertex), update it.

   Repeat this step V - 1 times (where V is the number of vertices). Here, V = 4, so we'll repeat it 3 times.

1st Iteration:

- **A → B: Distance to B = min(∞, 0 + 4) = 4**
- **A → C: Distance to C = min(∞, 0 + 5) = 5**
- **B → C: Distance to C = min(5, 4 + 3) = 5 (no change)**
- **C → D: Distance to D = min(∞, 5 + 4) = 9**

Note:Infinity (∞) is used to indicate that a vertex has not yet been reached from the source vertex, so its distance is unknown.

Bellman-Ford Algorithm

2nd Iteration:

Edge	Calculation	Updated?
A → B	min(4, 0 + 4) = 4	No
A → C	min(5, 0 + 5) = 5	No
B → C	min(5, 4 + 3) = 7	No
C → D	min(9, 5 + 4) = 9	No

Bellman-Ford Algorithm

3rd iteration:

A → B: (No update, already 4)min(4, 0 + 4) = 4

A → C: (No update, already 5)min(5, 0 + 5) = 5

B → C: (No update, already 7)min(5, 4 + 3) = 7

C → D: (No update, already 9)min(9, 5 + 4) = 9

A: 0

B: 4

C: 5

D: 9

3. Negative weight cycle detection:

No further updates, so there are no negative weight cycles.

Final Distances from A:

A: 0

B: 4

C: 5

D: 9

Memorization

- What is Memoization?

- Memoization is a **top-down** dynamic programming technique where we store the **results of expensive function calls** and return the **cached result** when the **same inputs** occur again.
- Describe the memorization Approach used in Dynamic Programming by illustrating the recursive Fibonacci function. Construct the control stack and illustrate the process of Memorization table.

```
int fibonacci(int n)
 { if(n == 0) return 0;
else if(n == 1) return 1;
else return (fibonacci(n-1) + fibonacci(n-2));
 }
```

Memorization

- Find the value of fibonacci(5) using memoization.
- _____
- 🔁 Step 1: Original (Recursive) Fibonacci (No Memoization)

```
int fibonacci(int n) {
   if(n == 0) return 0;
   if(n == 1) return 1;
   return fibonacci(n-1) + fibonacci(n-2);
}
```

- 🔴 This is slow because it recalculates values again and again.
- _____
- ✅ Step 2: Add Memoization (Store Already Computed Values)
- We will use an array fibMemo[] to store already calculated Fibonacci numbers.

Memorization


```
int fibMemo[100]; // assume n <= 99
int fibonacci(int n) {
if (fibMemo[n] != -1) // already calculated
 return fibMemo[n];
  if (n == 0) return fibMemo[n] = 0;
  if (n == 1) return fibMemo[n] = 1;
  fibMemo[n] = fibonacci(n-1) + fibonacci(n-2);
  return fibMemo[n];
}
```

Before calling this function, initialize all values to -1:

```
for (int i = 0; i < 100; i++)
  fibMemo[i] = -1;
```

💡 Example: Compute fibonacci(5)

It fills the table as:

Memorization

n	fib(n)
0	0
1	1
2	1
3	2
4	3
5	5 ✅

So, fibonacci(5) = 5

🎯 Final Answer:

fibonacci(5) returns 5 using memoization

Differnce b/n Dynamic And Divide and conquer

Aspect	Divide and Conquer	Dynamic Programming
Approach	Divide the problem into smaller subproblems, solve them independently, and combine their results.	Break the problem into overlapping subproblems and store their results to avoid recomputation.
Subproblem Overlap	Subproblems are generally non-overlapping.	Subproblems are overlapping (same subproblems are solved multiple times).
Optimal Substructure	Required	Required
Use of Memoization	Not used	Core concept (uses memoization or tabulation).
Examples	Merge Sort, Quick Sort, Binary Search, Strassen's Matrix Multiplication	Fibonacci Sequence, Knapsack Problem, Longest Common Subsequence
Efficiency	May solve the same subproblem multiple times (inefficient for overlapping subproblems).	Avoids redundant computation by storing intermediate results.
Recursion	Core technique, often used with recursion.	Often uses recursion with memorization or iteration with tabulation.


Thank
You